Project 2 - Digital Multimeter

Ray Lui
Professor Hummel
CPE 316 - 03
Computer Engineering Department
California Polytechnic State University, San Luis Obispo
Spring Quarter 5/22/2024

Behavior Description

The STM32L476 NUCLEO-L467RG Development Board was programmed to function as a digital multimeter in this project. The digital multimeter takes an input signal, AC or DC, as an input, and can measure frequency, peak-to-peak voltage, and true root mean square voltage for AC signals, and can measure DC voltage for DC signals. For both AC and DC signals, the digital multimeter is able to measure voltages from 0 - 3.3 V. As for the AC signals as inputs, the digital multimeter is able to read and measure sine, triangle, sawtooth, and square waves. The measured values for AC and DC signals are displayed neatly in the terminal through USART, with bar graphs for each voltage measurement. The digital multimeter is accurate within 10 mV for voltage measurements, and 1 Hz for frequency measurements.

System Specification

Parameter	Description
MCU	
Clock Frequency	24 MHz
DC Measurements	
DC Voltage Input Range	0 - 3.3 V
Voltage Measurement Error	± 10 mV
AC Measurements	
AC Voltage Input Range	0 - 3.3 V
Voltage Measurement Error	± 10 mV
Input Frequency	1 - 1000 Hz
Frequency Measurement Error	± 1 Hz
Input Peak-to-Peak Voltage	Min - 0.5 V
Input DC Offset	Max - 2.75 V
ADC	
Input Voltage	0 - 3.3 V
Clock Frequency	24 MHz
Bit Resolution	12 Bits
UART	
Baud Rate	Deci: 209; Hex: 0xD1
Bar Graph Units	10 mV per tick

Figure 1. System Specification Table

System Schematic

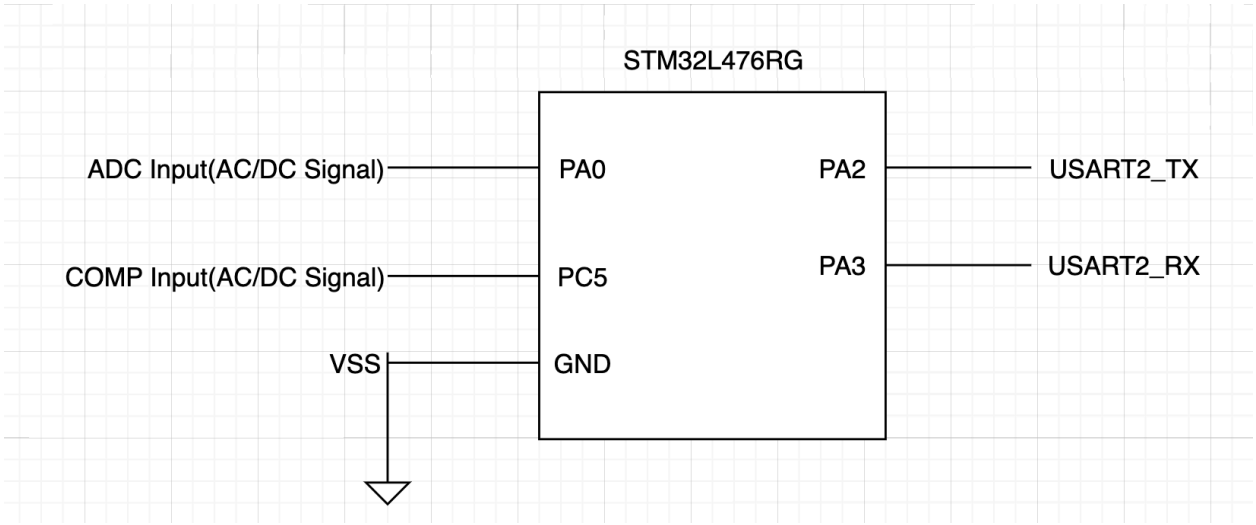

Figure 2. System Schematic

Software Architecture

The program starts by checking if the number of samples have been reached, and if it has, the program goes on to calculate AC or DC values, depending on whether or not the input signal is AC or DC. Then, the values are displayed in the terminal via UART, as seen in Figure 6.

Flowchart for Main Loop:

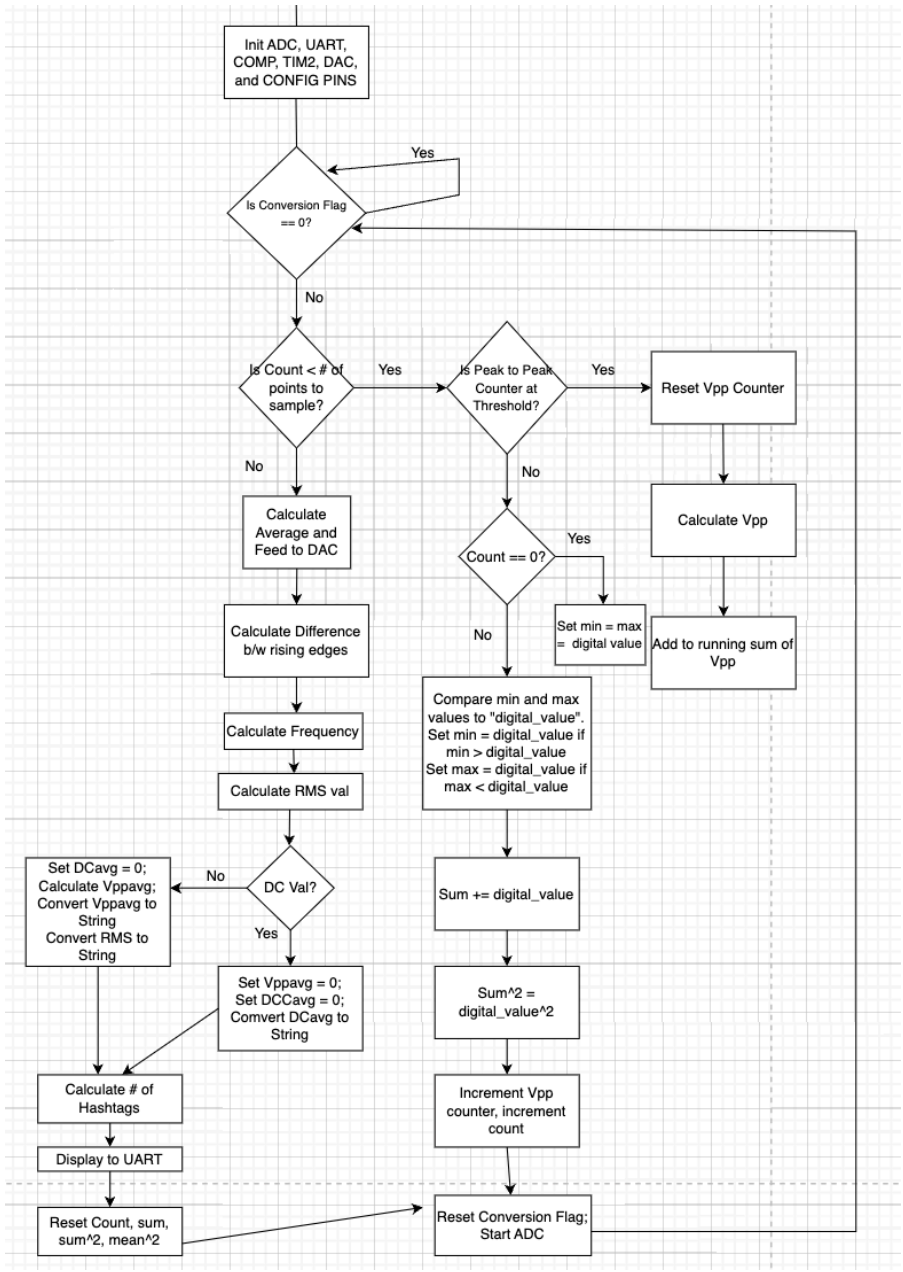

Figure 3. Main Loop Flowchart

Flowchart for TIM2_IRQHandler:

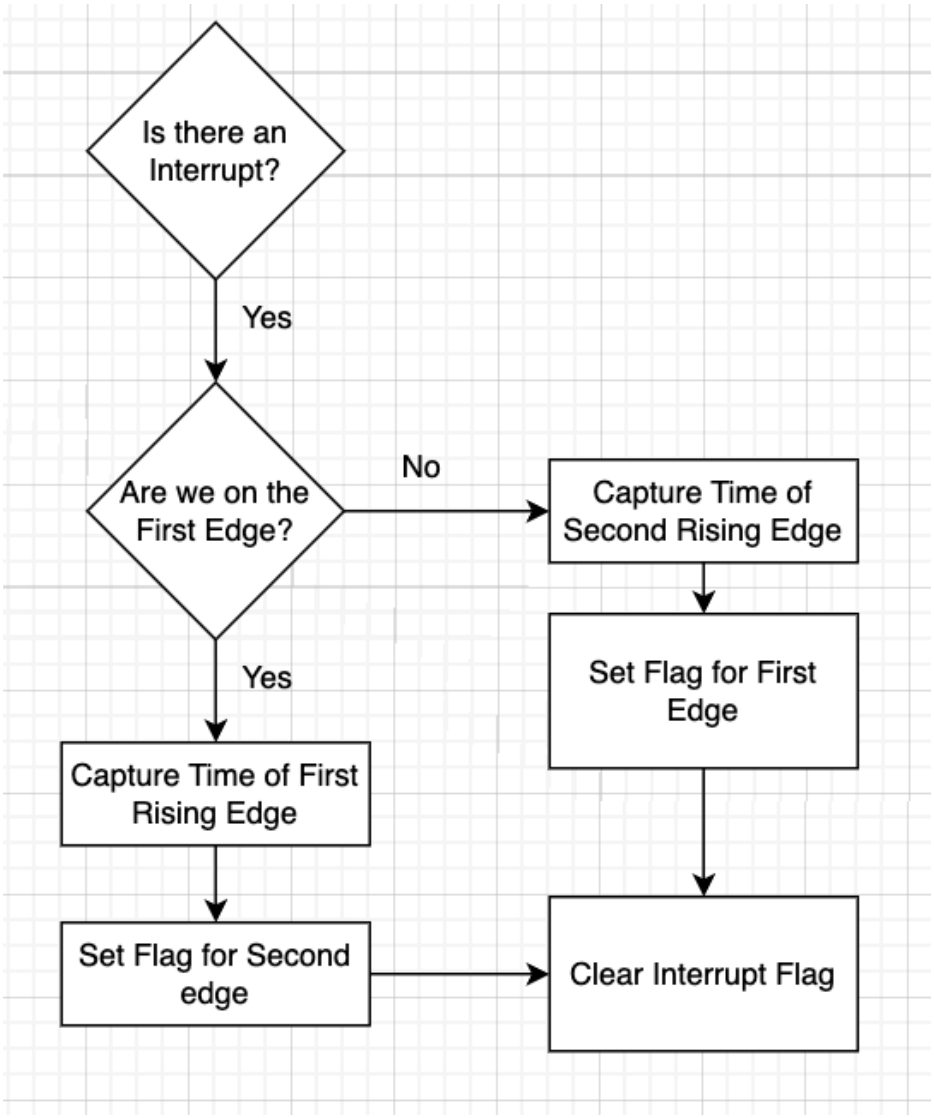

Figure 4. TIM2_IRQHandler Flowchart

Flowchart for ADC1_2_IRQHandler:

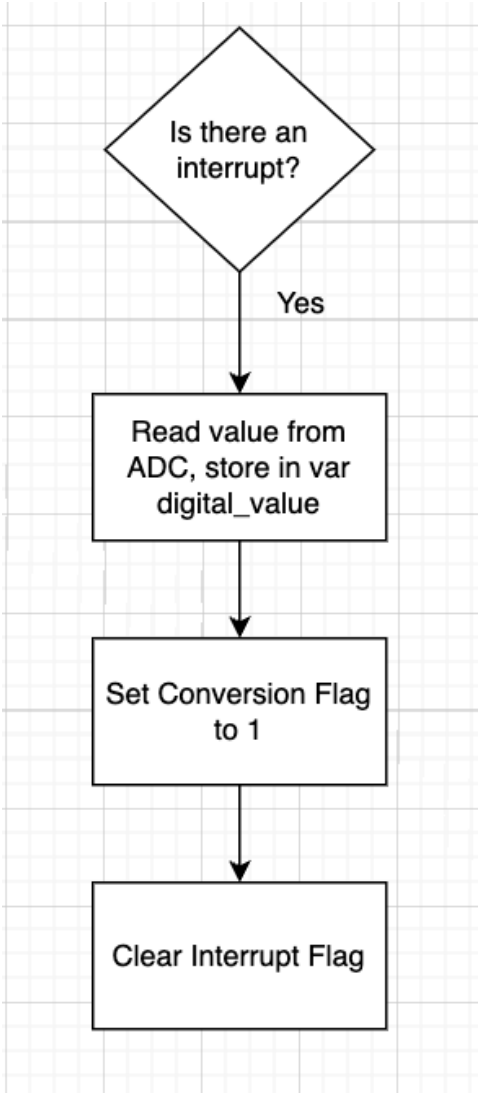

Figure 5. ADC1_2_IRQHandler Flowchart

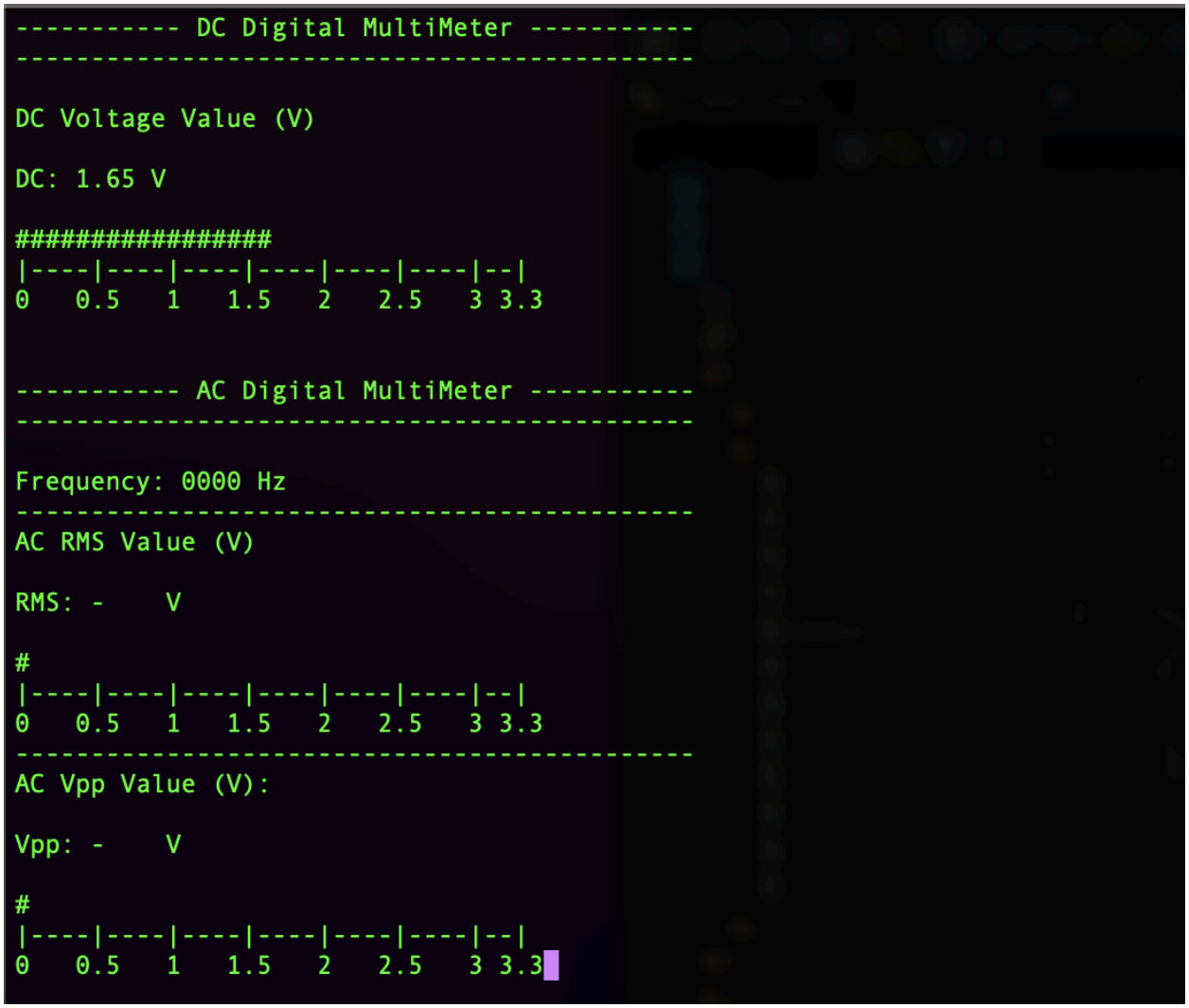

Figure 6. UART DMM Display

Appendices

main.c

—------ —----------

```
#define size 65536
#define PkPkLimit 8192
#define AvgBitsShift 16
#define DAC_MSK 0xFFF
#define inverter -1
#define CLK_FREQ 24000000
#define FreqStrLen 5
#define twoPercent 0.02
#define VppSamples 8
#define CalibrationMult 810
#define CalibrationAdd 10000
#define BarGraphTick 100000
#define MaxTicks 33

void SystemClock_Config(void);
void ConfigComparator(void);
void ConfigPin(void);
void ADC1_2_IRQHandler(void);
void ConfigDAC(void);
void ConfigTimer();

// global vars
uint16_t digital_value;
uint8_t conversionFlag;
uint8_t captureFlag = 0;
uint32_t FirstEdge;
uint32_t SecondEdge;

int main(void)
{
  HAL_Init();

  SystemClock_Config();

  UART_init();

  ConfigPin();

  ADC_init();

  ConfigComparator();

  ConfigDAC();

  ConfigTimer();
```

```
// init some vars
uint32_t count = 0;
uint32_t VppCounter = 0;
uint32_t avg = 0;
uint32_t DCavg = 0;
uint32_t sum = 0;
uint32_t freq;
int32_t difference;
uint32_t DCnum_hashtags = 0;
uint32_t Vppnum_hashtags = 0;
uint32_t RMSnum_hashtags = 0;
uint32_t Vppavg = 0;
uint32_t min;
uint32_t max;
uint32_t Vpp;
uint64_t SumSquared;
uint32_t meanSquared;
uint32_t RMS_val;

char freqStr[FreqStrLen];
char DCVoltageStr[5];
char VppStr[5];
char RMSStr[5];

while (1)
{
        // wait for conversion flag to be set
        while(conversionFlag != 1){
        }
        // check if we are done sampling
        if (count < size){

                // average Vpp values over 1/16 of a period
                if(VppCounter == PkPkLimit){
                        // reset the Vpp counter
                        VppCounter = 0;
                        // calculate this portion's Vpp
                        Vpp = GetVpp(min, max);
                        // add to our running sum to take avg
                        Vppavg += Vpp;
                }

                if(count == 0){
                        // set the min and max to first value we record
                        min = digital_value;
                        max = digital_value;
                }

                // set new max if current value greater than max
                if(digital_value > max){
                        max = digital_value;
                }
```

```
        // set new min if current value less than max
        if(digital_value < min){
                min = digital_value;
        }

        // add value we read from ADC
        sum += digital_value;

        // add to sum squared for RMS
        SumSquared += (digital_value * digital_value);

        // increment counters
        count += 1;
        VppCounter += 1;
}

else{
        // get average, shift right 13 bits
        avg = (sum >> AvgBitsShift);
        // 12 bit DAC
        DAC->DHR12R1 = (avg & DAC_MSK);

        // calculate difference
        difference = (SecondEdge - FirstEdge);
        // make difference positive if negative
        if(difference < 0){
                difference = (difference * inverter);
        }

        // calculate frequency
        freq = (uint32_t)(CLK_FREQ / difference);

        // calculate RMS value
        meanSquared = (uint32_t)(SumSquared >> AvgBitsShift);
        RMS_val = sqrt(meanSquared);

        // check for our DC threshold
        uint32_t TwoPerc = sum * twoPercent;
        uint32_t DCCheck = ((digital_value * count) + digital_value);

        // DC value -> set frequency to 0
        if((DCCheck <= (sum + TwoPerc)) && (DCCheck >= (sum - TwoPerc))){
                freq = 0;
        }

        // calibrate the freq based on levels
        freq = CalibrateFreq(freq);
        // convert freq to a string
        ConvertNum(freq, freqStr);

        // DC MODE
```

```
if(freq == 0){
        // calibrate the DC voltage
        DCavg = CalibrateDCVoltage(avg);
        // convert it to a string
        ConvertVolt(DCavg, DCVoltageStr);
        // empty out Vpp string buffer
        EmptyString(VppStr);
        EmptyString(RMSStr);
        // reset AC values
        Vppavg = 0;
        RMS_val = 0;
}

// AC MODE
else{
        // empty out DC voltage string buffer
        EmptyString(DCVoltageStr);
        // set DC avg to 0
        DCavg = 0;
        // divide by number of times we summed Vpp
        Vppavg = (Vppavg / VppSamples);
        // calibrate Vpp
        Vppavg = CalibrateVpp(Vppavg, freq);
        // calibrate RMS val
        RMS_val = (RMS_val * CalibrationMult) + CalibrationAdd;
        // convert Vpp and RMS to a string
        ConvertVolt(Vppavg, VppStr);
        ConvertVolt(RMS_val, RMSStr);
}

// get number of hashtags for our meter on DMM
RMSnum_hashtags = (RMS_val / BarGraphTick) + 1;

// check if num of hashtags exceed max
if(RMSnum_hashtags > MaxTicks){
        RMSnum_hashtags = 1;
}

DCnum_hashtags = (DCavg / BarGraphTick) + 1;
if(DCnum_hashtags > MaxTicks){
        DCnum_hashtags = 1;
}

Vppnum_hashtags = (Vppavg / BarGraphTick) + 1;
if(Vppnum_hashtags > MaxTicks){
        Vppnum_hashtags = 1;
}
USART_ESC_Code("[H");
// Display DMM screen
Display_DMM_Screen(freqStr, DCVoltageStr, DCnum_hashtags, Vppnum_hashtags,
                   VppStr, RMSStr, RMSnum_hashtags);
```

```
                // reset our values
                count = 0;
                sum = 0;
                SumSquared = 0;
                meanSquared = 0;
        }

        // set Flag to wait for another interrupt
        conversionFlag = 0;
        // start ADC again
        ADC1->CR |= (ADC_CR_ADSTART);
    }
 }

void ConfigComparator(void){

       // set input to PC5
       COMP1->CSR &= ~(COMP_CSR_INPSEL);
       // set the minus input of our comparator
       COMP1->CSR |= (4 << COMP_CSR_INMSEL_Pos);
       // set to high hysteresis
       COMP1->CSR |= (3 << COMP_CSR_HYST_Pos);
       // enable the V(ref) Scaler
       COMP1->CSR |= COMP_CSR_SCALEN;
       // enable the comparator
       COMP1->CSR |= COMP_CSR_EN;
       // set to ultra low power
       COMP1->CSR |= (COMP_CSR_PWRMODE);

}

void ConfigPin(void){

       // enable clock for GPIOC
       // PC5 input to comparator
       RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOCEN);
       // configure PC5 input to comparator
       // PC5 to analog mode
       GPIOC->MODER &= ~(GPIO_MODER_MODE5);
       GPIOC->MODER |= (3 << GPIO_MODER_MODE5_Pos);
       // set PC5 to push pull mode
       GPIOC->OTYPER &= ~(GPIO_OTYPER_OT5);
       // set PC5 to low speed
       GPIOC->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED5);
       // set to no resistor for PC5
       GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD5);

}

void ConfigDAC(void){

       // Config the GPIO PB0 for the COMP OUTPUT
```

```
        RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
        // CLEAR MODE
        GPIOB->MODER &= ~(GPIO_MODER_MODE0);
        // Set to AF mode
        GPIOB->MODER |= GPIO_MODER_MODE0_1;
        // set AF to AF 12
        GPIOB->AFR[0] |= (GPIO_AFRL_AFSEL0_3 | GPIO_AFRL_AFSEL0_2);
        // Config the DAC
        RCC -> APB1ENR1 |= RCC_APB1ENR1_DAC1EN;
        // DAC channel 1 to comparator with buffer
        DAC->MCR &= ~(DAC_MCR_MODE1);
        DAC->MCR |= (1 << DAC_MCR_MODE1_Pos);
        //enable the DAC
        DAC->CR |= DAC_CR_EN1;

}

void ConfigTimer(){

        // enable TIM2 clock
        RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;
        // use TIM2 input capture 4 - tied to COMP output
        // clear CC1S
        TIM2->CCMR2 &= ~(TIM_CCMR2_CC4S);
        // TIM2_CCR1 must be linked to the TI1 input
        TIM2->CCMR2 |= (1 << TIM_CCMR2_CC4S_Pos);
        // clear filter bit
        TIM2->CCMR2 &= ~(TIM_CCMR2_IC4F);
        // set to high filter
        TIM2->CCMR2 |= (8 << TIM_CCMR2_IC4F_Pos);
        // set to upcount mode
        TIM2->CR1 &= ~(TIM_CR1_DIR);
        // connect to output of COMP1
        TIM2->OR1 |= TIM2_OR1_TI4_RMP_0;
        // enable capture
        TIM2->CCER |= TIM_CCER_CC4E;
        // enable the interrupt
        TIM2->DIER |= TIM_DIER_CC4IE;
        // now set the NVIC
        NVIC->ISER[0] = (1 << (TIM2_IRQn & 0x1F));
        // set global enable
        __enable_irq();
        // enable the clock
        TIM2->CR1 |= (TIM_CR1_CEN);

}

void TIM2_IRQHandler(void)
{
        // check if interrupt flag is ON
        if(TIM2->SR & TIM_SR_CC4IF){
                // check if we are on the first edge
```

```
                if(captureFlag == 0){
                        // capture the rising edge time
                        FirstEdge = TIM2->CCR4;
                        // set flag to capture next edge
                        captureFlag = 1;
                }
                // check if we are on second edge
                else if(captureFlag == 1){
                        // capture the rising edge time
                        SecondEdge = TIM2->CCR4;
                        // set flag to capture next edge
                        captureFlag = 0;
                }
                // clear interrupt flag
                TIM2->SR &= ~(TIM_SR_CC4IF);
        }

}
// ADC ISR
void ADC1_2_IRQHandler(void){

    if(ADC1->ISR & ADC_ISR_EOC){
        digital_value = ADC1->DR; // read data
        conversionFlag = 1;
    }
    ADC1->ISR &= ~(ADC_ISR_EOC);

}
```

UART.h

```
#include "stm32l4xx_hal.h"
#include <stdio.h>
#include <stdint.h>

#ifndef SRC_UART_H_
#define SRC_UART_H_

void UART_init(void);
void UART_print_char(char letter);
void UART_print_string(char *string);
void Display_DMM_Screen(char* freqStr, char* DCVoltStr, uint32_t DCnum_hashtags, uint32_t
                        Vppnum_hashtag, char* VppStr, char* RMSStr, uint32_t RMSnum_hashtags);

#endif /* SRC_UART_H_ */
```

UART.c

```c
#include "UART.h"
#define ESC_CODE 0x1b

void UART_init(void){
        // enable clock for GPIOA
        RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN);
        // enable the USART2 Clock
        RCC->APB1ENR1 |= RCC_APB1ENR1_USART2EN;

        // clear the AFR Bits
        GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL2 | GPIO_AFRL_AFSEL3);
        // set to alternate mode for USART2
        GPIOA->AFR[0] |= ((7 << GPIO_AFRL_AFSEL2_Pos) | (7 << GPIO_AFRL_AFSEL3_Pos));
        // clear mode bits
        GPIOA->MODER &= ~(GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
        // set to alternate function mode
        GPIOA->MODER |= ((2 << GPIO_MODER_MODE2_Pos) | (2 << GPIO_MODER_MODE3_Pos));
        // Push-pull output
        GPIOA->OTYPER &= ~(GPIO_OTYPER_OT3 | GPIO_OTYPER_OT2);
        // no resistor
        GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD3 | GPIO_PUPDR_PUPD2);
        // low speed
        GPIOA->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED3 | GPIO_OSPEEDR_OSPEED2);

        // set word length to 8 bits
        USART2->CR1 &= ~(USART_CR1_M0);
        USART2->CR1 &= ~(USART_CR1_M1);
        // Set Over 8 bit to 0
        USART2->CR1 &= ~(USART_CR1_OVER8);
        // Set Baud Rate to Deci: 209, Hex: 0xD1
        USART2->BRR = 0xD1;
        // Set stop bit to 1 stop bit
        USART2->CR2 &= ~(USART_CR2_STOP);
        // disable parity bit
        USART2->CR1 &= ~(USART_CR1_PCE);
        // enable TE bit
        USART2->CR1 |= USART_CR1_TE;
        // enable the RE bit
        USART2->CR1 |= USART_CR1_RE;

        // enable USART
        USART2->CR1 |= USART_CR1_UE;
        // enable the interrupt RXNEIE later for ECHO I think
        USART2->CR1 |= USART_CR1_RXNEIE;
        // set NVIC -> POSITION 38
        NVIC->ISER[1] |= (1 << (USART2_IRQn & 0x1F));
        // set global enable
  __enable_irq();
}
```

```c
void UART_print_char(char letter){
	// wait for reg to be ready
	while (!(USART2->ISR & USART_ISR_TXE)){

	}
	// write the data
	USART2->TDR = letter;
	return;
}

void UART_print_string(char *string){
	// write all letters in the string
	for(int i = 0; (string[i] != '\0'); i++){
		UART_print_char(string[i]);
	}
}

void USART_ESC_Code(char* string){
	// write the ESC key first
	UART_print_char(ESC_CODE);
	// write the string for each escape command
	UART_print_string(string);
}


void Display_DMM_Screen(char* freqStr, char* DCVoltStr, uint32_t DCnum_hashtags, uint32_t
Vppnum_hashtag, char* VppStr, char* RMSStr, uint32_t RMSnum_hashtags){
	// write ESC code to move cursor to top left
	USART_ESC_Code("[H");
	UART_print_string("----------- DC Digital MultiMeter -----------");
	USART_ESC_Code("E");
	UART_print_string("---------------------------------------------");
	USART_ESC_Code("E");
	USART_ESC_Code("E");

	// PRINT DC VALUE
	UART_print_string("DC Voltage Value (V)");
	USART_ESC_Code("E");
	USART_ESC_Code("E");
	UART_print_string("DC: ");
	UART_print_string(DCVoltStr);
	UART_print_string(" V      ");
	USART_ESC_Code("E");
	USART_ESC_Code("E");

	// PRINT OUT BAR GRAPH
	USART_ESC_Code("[2K");
	USART_ESC_Code("[2K");
	for (int i = 0; i < DCnum_hashtags; i++){
		UART_print_string("#");
	}
```

```
USART_ESC_Code("E");
UART_print_string("|----|----|----|----|----|----|--|");
USART_ESC_Code("E");
UART_print_string("0   0.5   1   1.5   2   2.5   3 3.3");
USART_ESC_Code("E");
USART_ESC_Code("E");
USART_ESC_Code("E");

// PRINT AC
UART_print_string("----------- AC Digital MultiMeter -----------");
USART_ESC_Code("E");
UART_print_string("---------------------------------------------");
USART_ESC_Code("E");
USART_ESC_Code("E");

// PRINT FREQ
UART_print_string("Frequency: ");
UART_print_string(freqStr);
UART_print_string(" Hz");
USART_ESC_Code("E");
UART_print_string("---------------------------------------------");
USART_ESC_Code("E");

// PRINT RMS
UART_print_string("AC RMS Value (V)");
USART_ESC_Code("E");
USART_ESC_Code("E");
UART_print_string("RMS: ");
UART_print_string(RMSStr);
UART_print_string(" V");
USART_ESC_Code("E");
USART_ESC_Code("E");

// PRINT BAR GRAPH
USART_ESC_Code("[2K");
USART_ESC_Code("[2K");
for (int i = 0; i < RMSnum_hashtags; i++){
       UART_print_string("#");
}
USART_ESC_Code("E");
UART_print_string("|----|----|----|----|----|----|--|");
USART_ESC_Code("E");
UART_print_string("0   0.5   1   1.5   2   2.5   3 3.3");
USART_ESC_Code("E");
UART_print_string("---------------------------------------------");
USART_ESC_Code("E");

// PRINT Vpp
UART_print_string("AC Vpp Value (V): ");
USART_ESC_Code("E");
USART_ESC_Code("E");
```

```
        UART_print_string("Vpp: ");
        UART_print_string(VppStr);
        UART_print_string(" V");
        USART_ESC_Code("E");
        USART_ESC_Code("E");
        USART_ESC_Code("[2K");
        USART_ESC_Code("[2K");

        // PRINT BAR GRAPH
        for (int i = 0; i < Vppnum_hashtag; i++){
                UART_print_string("#");
        }
        USART_ESC_Code("E");
        UART_print_string("|----|----|----|----|----|----|--|");
        USART_ESC_Code("E");
        UART_print_string("0   0.5   1   1.5   2   2.5   3 3.3");

}
```

ADCCalculations.h

```
#include "stm32l4xx_hal.h"
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

#ifndef SRC_ADCCALCULATIONS_H_
#define SRC_ADCCALCULATIONS_H_

void ConvertNum(uint32_t num, char* string);
uint32_t CalibrateDCVoltage(uint32_t voltage);
void ConvertVolt(uint32_t num, char* string);
uint32_t GetVpp(uint32_t min, uint32_t max);
uint32_t CalibrateFreq(uint32_t freq);
uint32_t CalibrateVpp(uint32_t Vpp, uint32_t freq);
void EmptyString(char* String);

#endif /* SRC_ADCCALCULATIONS_H_ */
```

ADCCalculations.c

```c
#include "ADCCalculations.h"

void ConvertNum(uint32_t num, char* string){
        // Divisors to get the first 4 digits
        uint32_t divisor0 = 1000;
        uint32_t divisor1 = 100;
        uint32_t divisor2 = 10;
        uint32_t divisor3 = 1;
        // Place holders for nums
        uint8_t digit0 = (uint8_t)((num / divisor0) % 10);
        uint8_t digit1 = (uint8_t)((num / divisor1) % 10);
        uint8_t digit2 = (uint8_t)((num / divisor2) % 10);
        uint8_t digit3 = (uint8_t)((num / divisor3) % 10);

    // Convert digits to characters and store them in the string

    // Convert digit0 to character and store at index 0
    string[0] = digit0 + '0';
    // Convert digit1 to character and store at index 1
    string[1] = digit1 + '0';
    // Convert digit1 to character and store at index 1
    string[2] = digit2 + '0';
    // Convert digit3 to character and store at index 3
    string[3] = digit3 + '0';
    // null char
    string[4] = '\0';
}

void ConvertVolt(uint32_t num, char* string){
        // Divisors to get the first 4 digits
        uint32_t divisor0 = 1000000;
        uint32_t divisor1 = 100000;
        uint32_t divisor2 = 10000;
        // Place holders for nums
        uint8_t digit0 = (uint8_t)((num / divisor0) % 10);
        uint8_t digit1 = (uint8_t)((num / divisor1) % 10);
        uint8_t digit2 = (uint8_t)((num / divisor2) % 10);

    // Convert digits to characters and store them in the string

    // Convert digit0 to character and store at index 0
    string[0] = digit0 + '0';
    // "." for voltages
    string[1] = '.';
    // Convert digit1 to character and store at index 1
    string[2] = digit1 + '0';
    // Convert digit2 to character and store at index 2
```

```
    string[3] = digit2 + '0';
    // null char
    string[4] = '\0';
}

uint32_t CalibrateDCVoltage(uint32_t voltage){
        // this returns the output voltage in micro volts (uV)
        // values used in this func are for calibration
        // for certain voltage ranges

        uint32_t output = (uint32_t)(810 * voltage);

        if (output < 150000){
                output = output + 30000;
        }
        else if((output < 500000) && (output > 150000)){
                output = output + 10000;
        }
        else if((output < 700000) && (output > 500000)){
                output = output + 10000;
        }
        else if((output < 1250000) && (output > 1000000)){
                output = output - 10000;
        }
        else if ((output < 2000000) && (output > 1500000)){
                output = output - 20000;
        }
        else if ((output < 2500000) && (output > 2000000)){
                output = output - 30000;
        }
        else if ((output < 2680000) && (output > 2500000)){
                output = output - 40000;
        }
        else if((output < 3000000) && (output > 2680000)){
                output = output - 65000;
        }
        else if ((output < 4500000) && (output > 3000000)){
                output = output - 70000;
        }
//      output = (output - 20000);
        return output;
}

uint32_t CalibrateVpp(uint32_t Vpp, uint32_t freq){
        // values used in this func are for calibration
        // for certain voltage ranges
        if((Vpp < 620000) && (Vpp > 400000)){
                Vpp = Vpp - 40000;
        }
        else if((Vpp < 1200000) && (Vpp > 620000)){
```

```
                Vpp = Vpp - 60000;
        }
        else if((Vpp < 1520000) && (Vpp > 1200000)){
                Vpp = Vpp - 100000;
        }
        else if((Vpp < 2200000) && (Vpp > 1520000)){
                Vpp = Vpp -140000;
        }
        else if((Vpp < 2500000) && (Vpp > 2200000)){
                Vpp = Vpp -100000;
        }
        else if((Vpp < 2900000) && (Vpp > 2500000)){
                Vpp = Vpp -220000;
        }
        else if((Vpp < 3100000) && (Vpp > 2900000)){
                Vpp = Vpp -220000;
        }
        else if((Vpp < 3700000) && (Vpp > 3100000)){
                Vpp = Vpp -230000;
        }
        return Vpp;
}

uint32_t GetVpp(uint32_t min, uint32_t max){
        // This func gets the Vpp value and
        // calibrates the values
        uint32_t Vpp = (max - min);
        Vpp = CalibrateDCVoltage(Vpp);
        Vpp = Vpp - 60000;
        return Vpp;
}

uint32_t CalibrateFreq(uint32_t freq){
          // calibrate the freq based on ranges of freqs
          if(freq > 200){
                  freq = freq + 2;
          }

          if(freq > 300){
                  freq = freq + 2;
          }

          if(freq > 500){
                 freq = freq + 1;
          }

          if(freq > 800){
                 freq = freq + 1;
          }
          if(freq > 900){
```

```
                freq = freq + 1;
            }
            return freq;
}

void EmptyString(char* String){
        // this func removes chars in strings
        String[0] = '-';
        String[1] = ' ';
        String[2] = ' ';
        String[3] = ' ';
    String[4] = '\0';
}
```

References

[1] Hummel, Paul, et al. "Microcontrollers and Embedded Applications Laboratory Manual."

[2] "STM32L4 Reference Manual." STM, Jan. 2024.

[3] "STM32L476 Datasheet." STM, Mar. 2024.